

Transport layer : (class)

Introduction | what is this?

Process to process → figure → node to node

figure
Port types, socket address

Multiplexing and demultiplexing

Connection oriented, connectionless (Important)

position of UDP, TCP, SCTP

TCP vs UDP * TCP services

Stream delivery service → Sending and receiving buffer

TCP segments, Example **

↓ Format, 6 flag

TCP connection — three way handshaking

6-8 mark

Question: three way handshaking, data transfer figure will be given, Acknowledgment and sequence number

will be hidden, we have to find them, socket address?, multiplexing?, demux?

Stream delivery ~~কেন~~ process to process

→ delivery ~~কেন~~ data link layer flow

control error control ~~কেন~~ process

to process A transmit layer A error control

~~কেন~~ ~~কেন~~

Error control: checksum mathx

↳ Concept

Acknowledgement, Retransmission

Three duplicate ACK segment

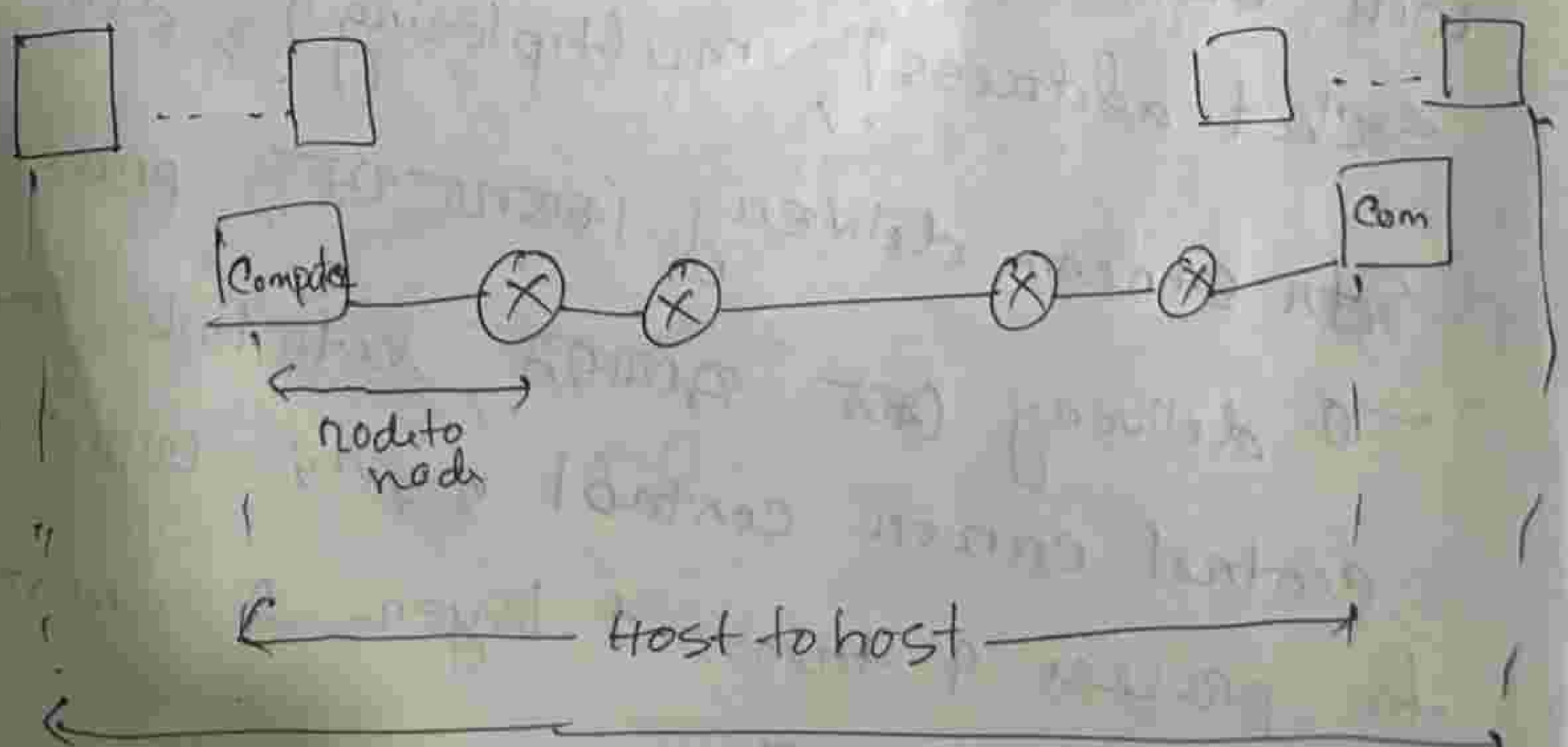
Lost segment

Transport Layer:

Node to node delivery: data link layer is responsible for delivery of frames between two neighboring nodes over a link

⇒ network layer is responsible for delivery of datagrams between two hosts. (host to host)

⇒ Two processes communicate in a client/server relationship



Here we need transport layer address.

a port number

→ to choose among multiple processes running on the destination

→ destination port number needed for delivery

→ Source C C C C reply

Port numbers are int (0-65535)

Epimeral port no: client program defines itself with a port number chosen randomly by the transport layer sw running on the client host.

→ server process must also define itself with a port number. This port number can't be chosen randomly.

Port number have 3 range

① We'll know ports: The ports ranging from 0 to 1023 assigned and controlled by IANA.

→ Reserved for most universal application

HTTP $\rightarrow$ 80

POP3 $\rightarrow$ 110

Registered Ports: ports ranging from 1024 to 49151 are not assigned or controlled.

App using these ports run as ordinary programs that can be started by non-privileged users.

Dynamic Ports: (49152 - 65535) neither controlled nor registered. σ

→ They can be used by any process. Ephemeral ports

Socket address:

Process to process needs two identifiers

① IP address

② Port number

Combination of IP address and port numbers are socket address.

Transport layer protocol needs a pair of socket addresses —

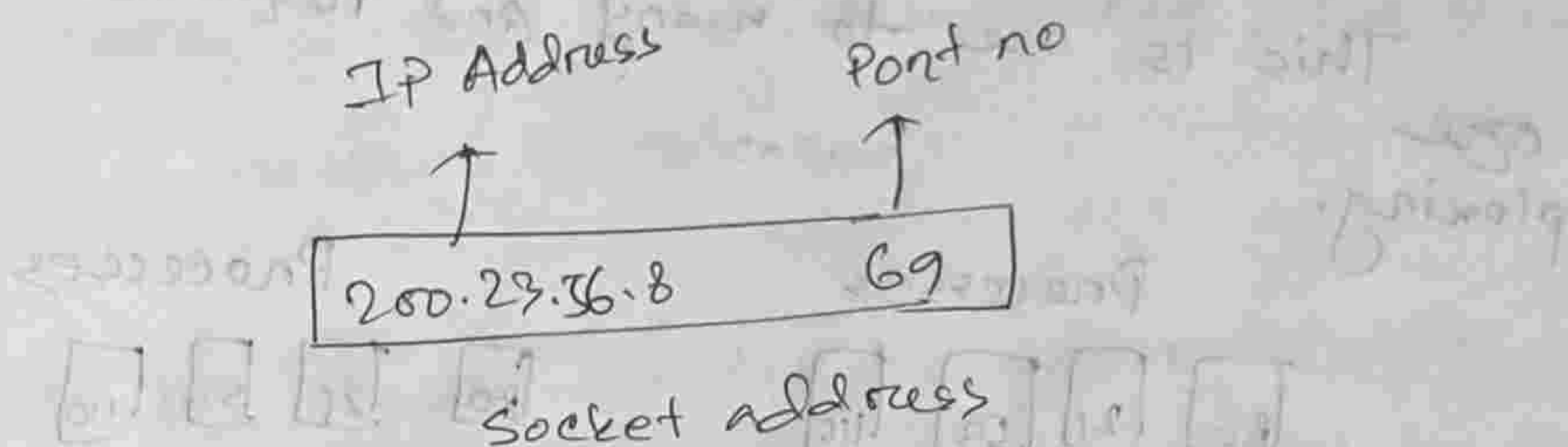
① Client socket address

② Server socket address.

This four pieces of information are part of the IP header and the transport layer

protocol header. IP header contains the IP address.

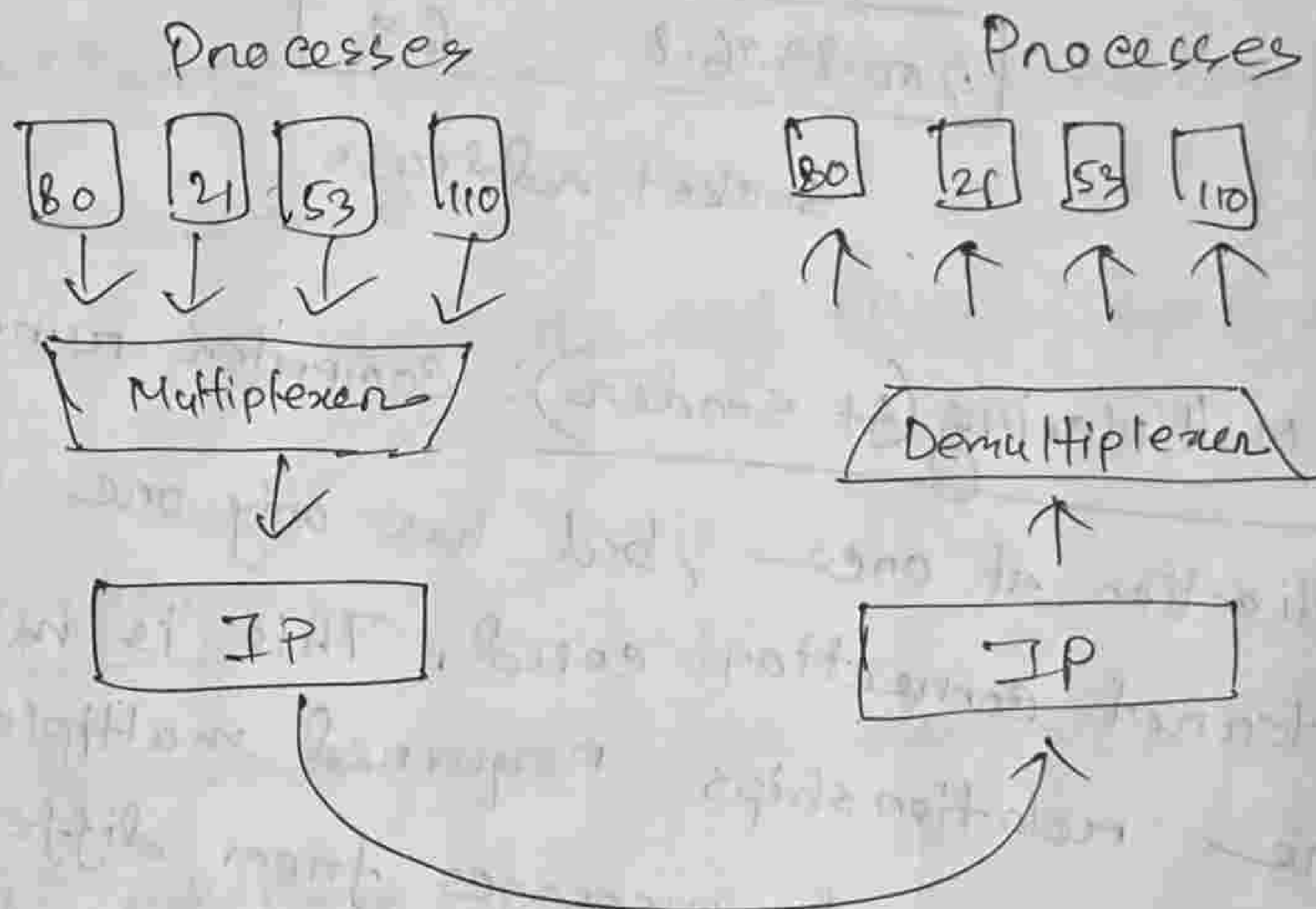
→ UDP and TCP header contains the port number.



Multiplexing (At sender): Computer runs multiple application at once, but has only one physical internet connection card. This is many to one relationships required multiplexing. The protocol accepts messages from different processes differentiated by their assigned port numbers. After adding the header, the transport layer passes the packet to the network layer.

Demultiplexing (At receiver): Computer receives a large stream of mixed packets from the network layer, the transport layer acts

as a sorting agent. It strips off the header reads the destination port numbers, and funnels each packet to the proper application window. This is one to many and required demultiplexing.



Connection Less Services: the packets are sent from one party to another with no need for connection establishment or connection release.

- packets are not numbered, they may be delayed or lost or may arrive out of sequence.
- No acknowledgement either.

Example — UDP

Connection Oriented Service: a connection is

first established between the sender and the receiver. Data are transferred, At the end connection is released.

⇒ If the application layer needs reliability, we use a reliable transport layer protocol by implementing flow and error control at the transport layer. This means a slower and more complex service.

⇒ If the application program doesn't need reliability because it needs fast service or the nature of the service doesn't demand flow and error control.

⇒ In the internet, 3 common different transport layer

① UDP is connectionless and unreliable

② TCP and SCTP are connection oriented and reliable.

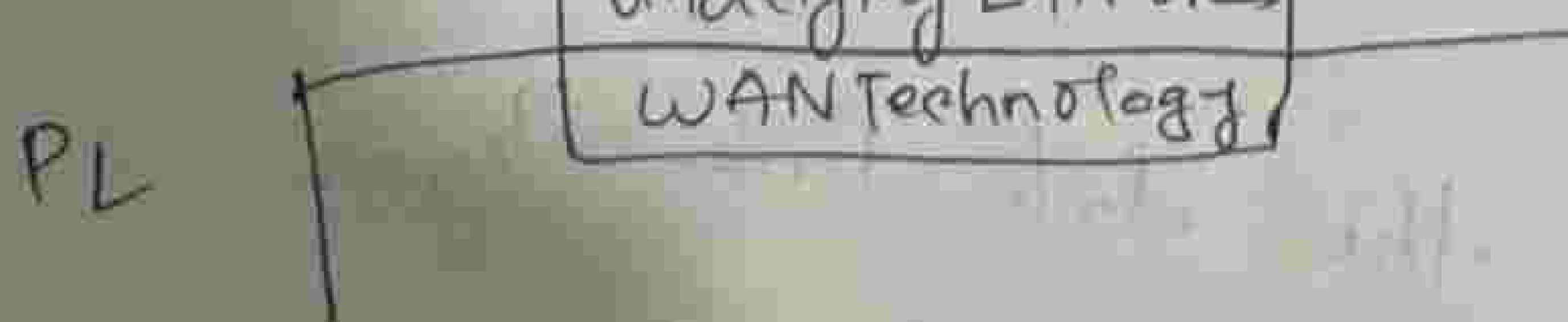
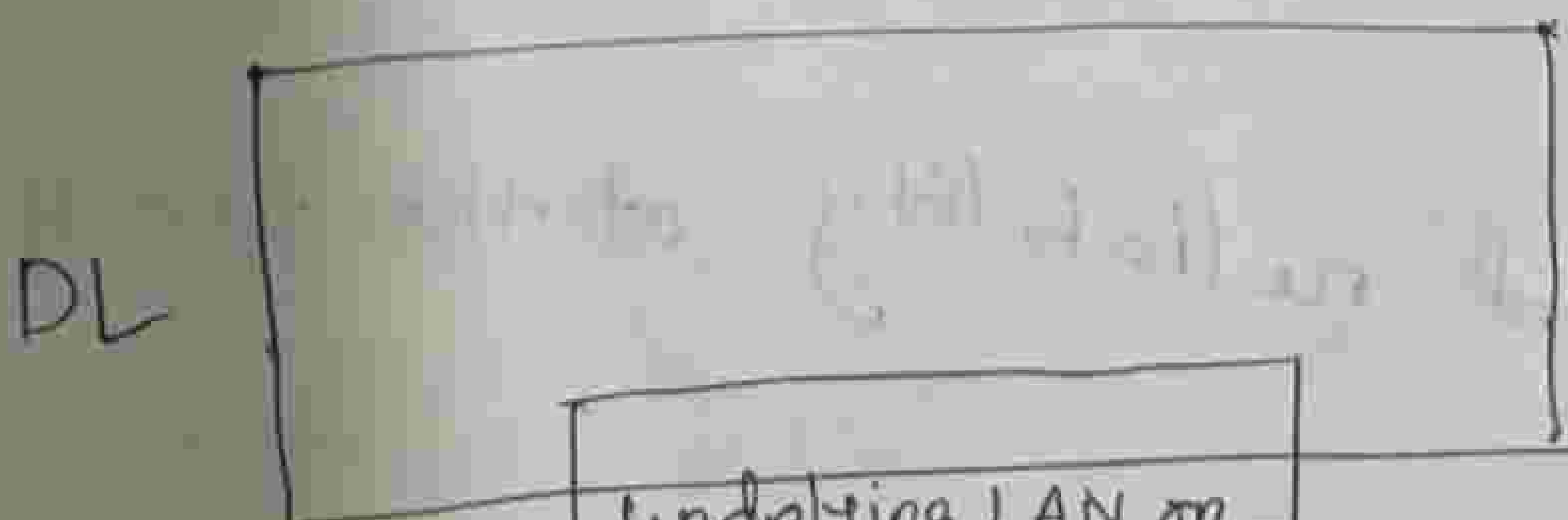
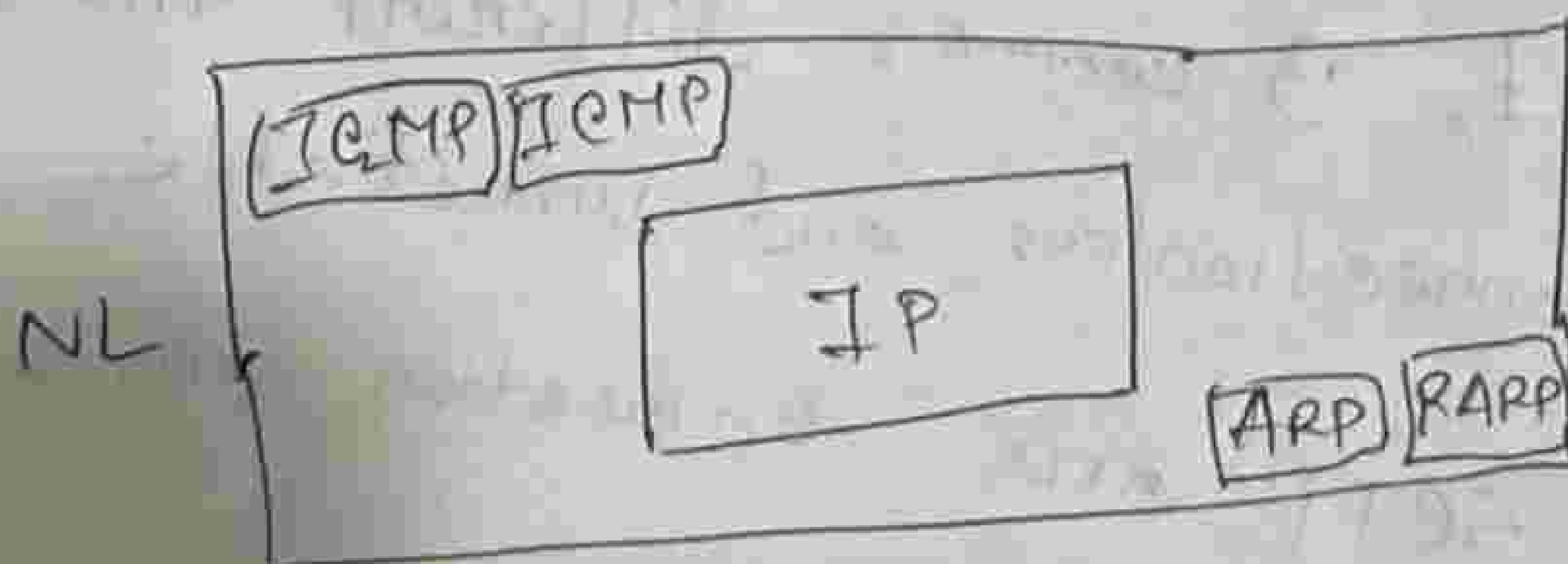
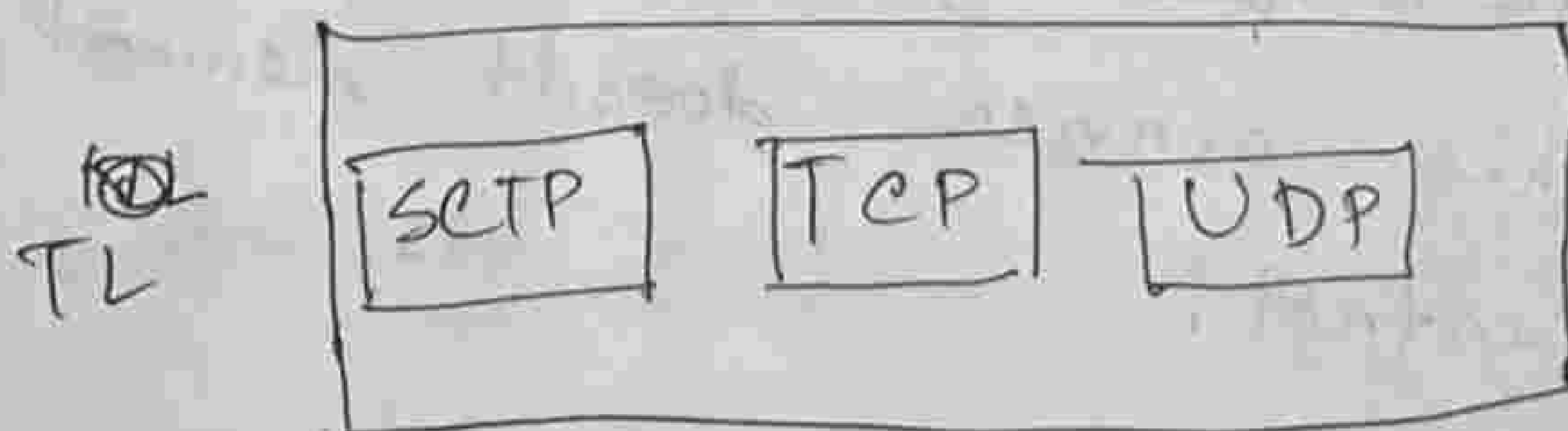
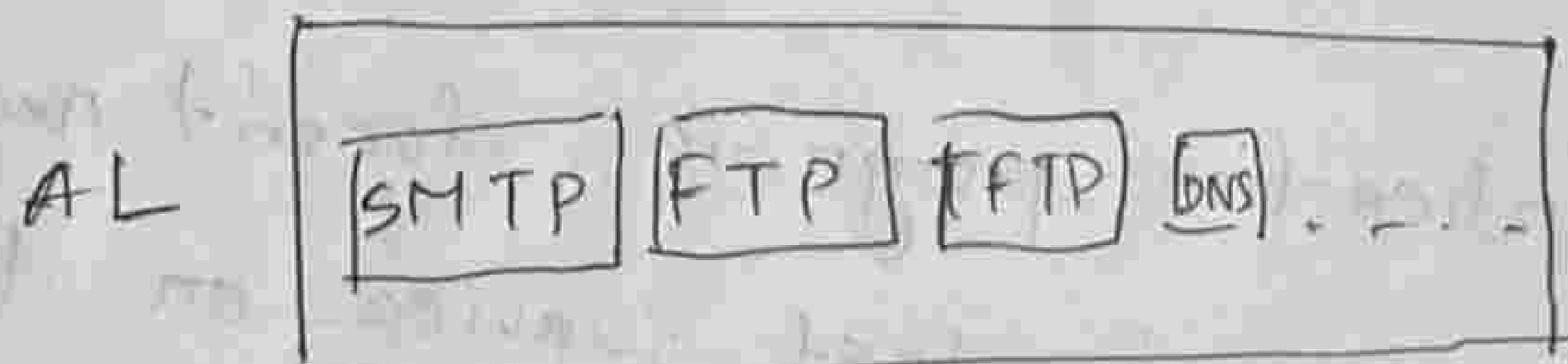
Why do we need reliability at the transport layer too?

Reliability at the data link layer is between

two nodes, we need reliability between the ends.

Because the network layer in the internet layer is unreliable, we need to implement reliability at the transport layer. To understand that error control at the data link layer does not guarantee error control at the transport layer.

Position: User Datagram protocol, Transmission control Protocol, Stream control transmission protocol

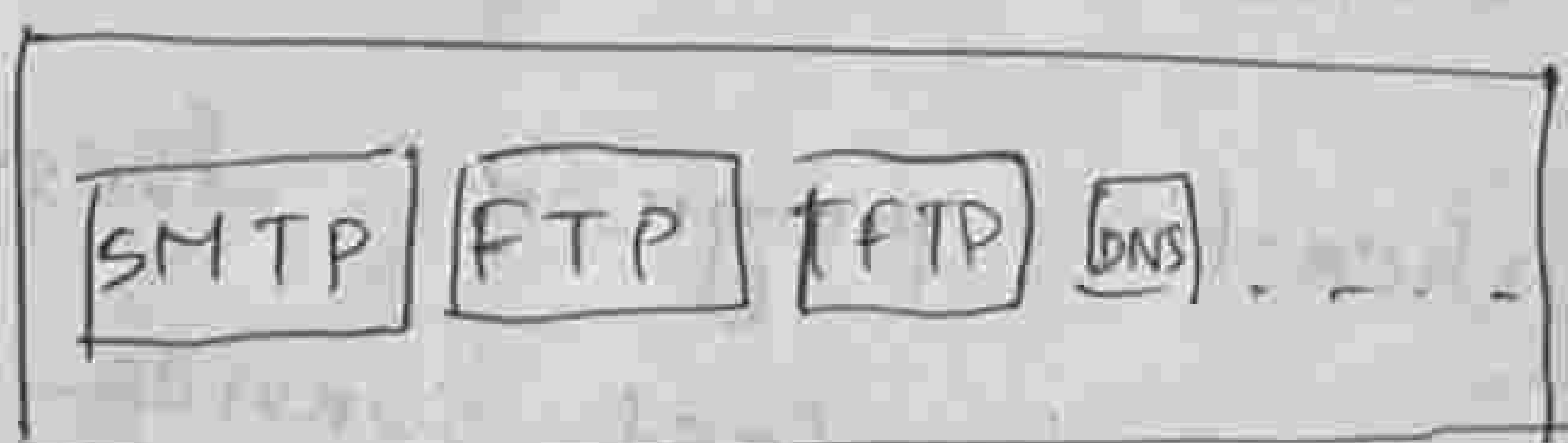


two nodes, we need reliability between the ends.

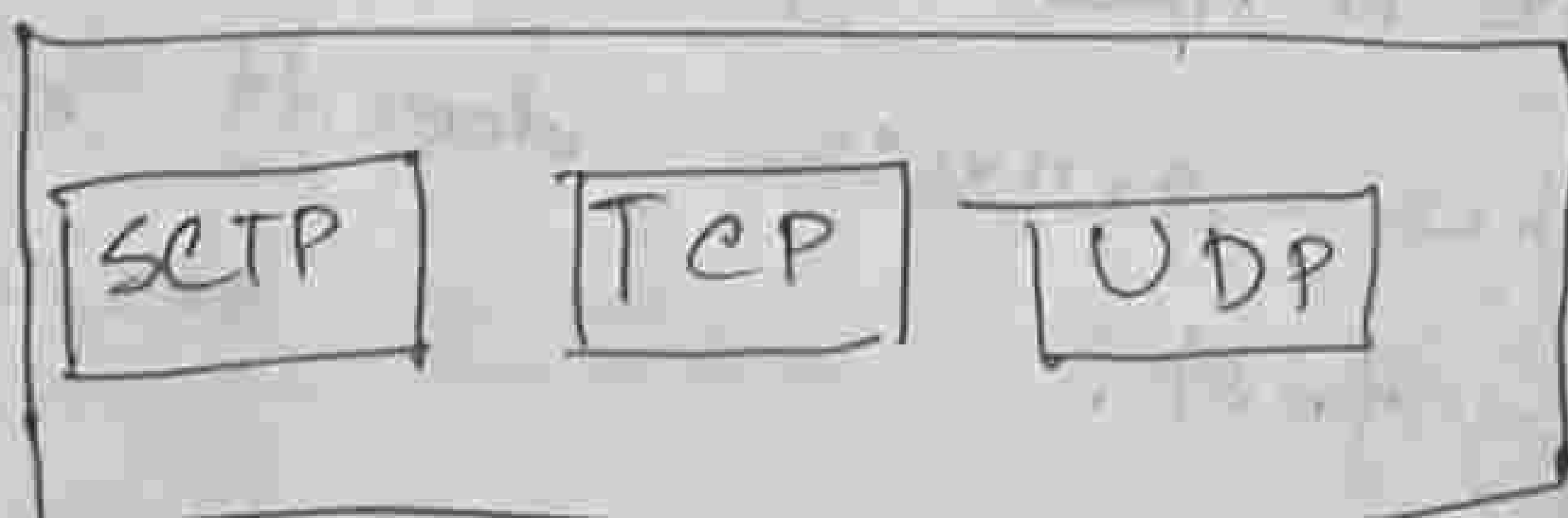
Because the network layer in the internet layer is unreliable, we need to implement reliability at the transport layer. To understand that error control at the data link layer does not guarantee error control at the transport layer.

Position: User Datagram protocol, Transmission control Protocol, Stream control transmission protocol

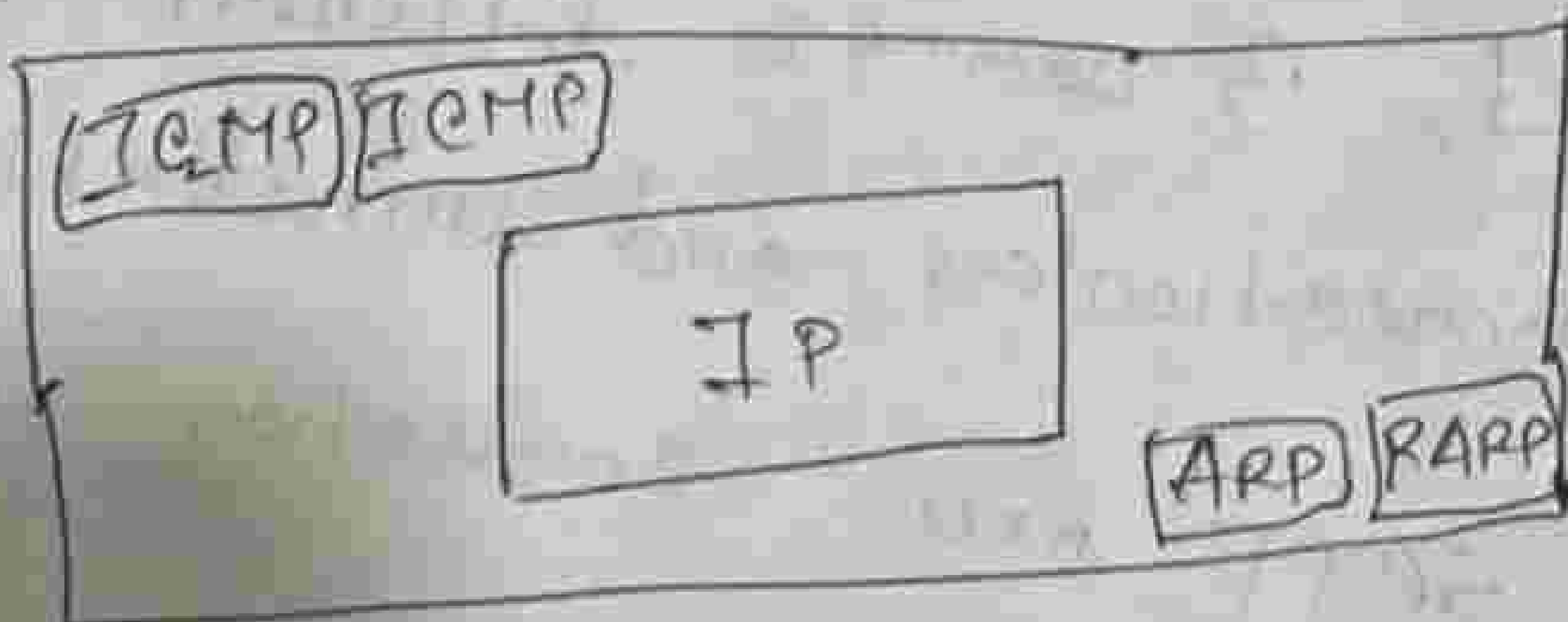
AL



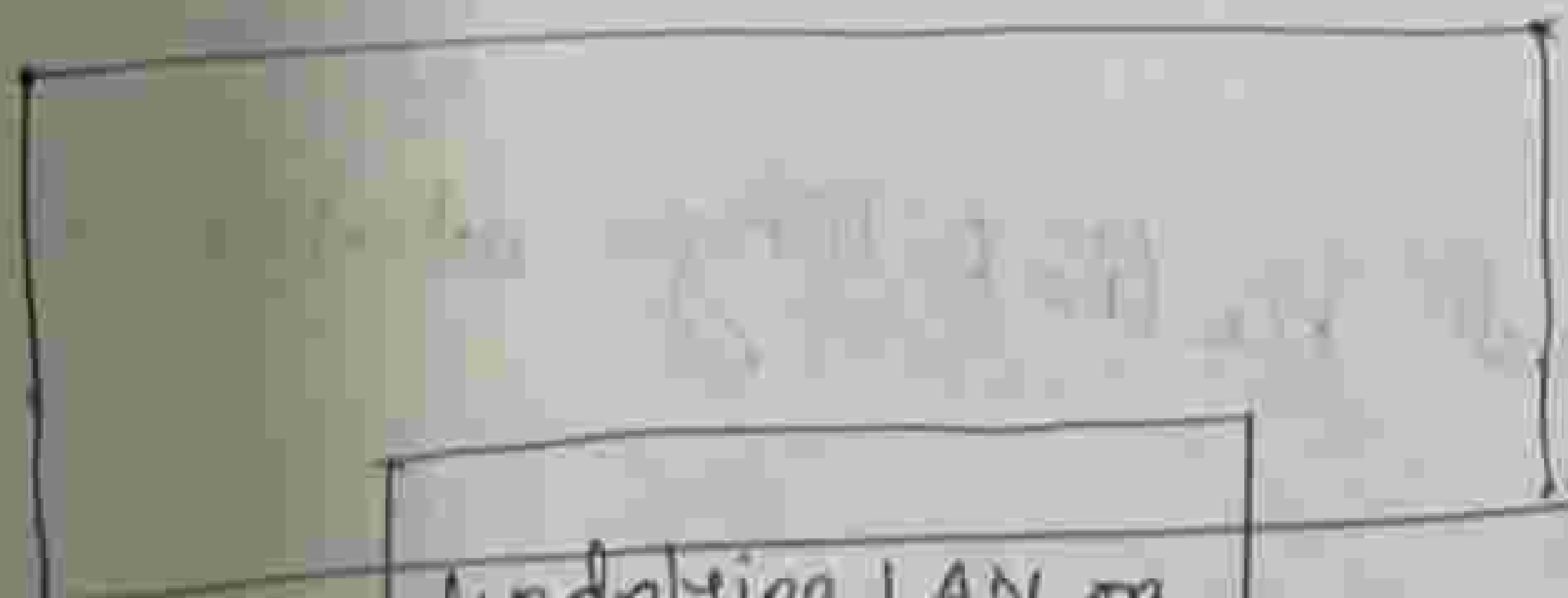
TL



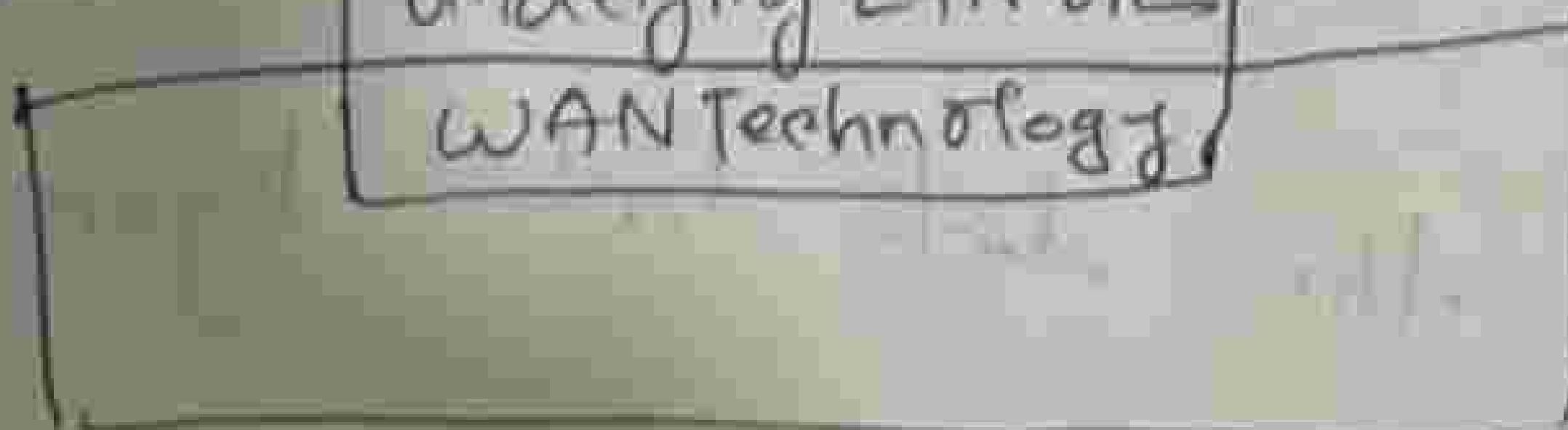
NL



DL



PL



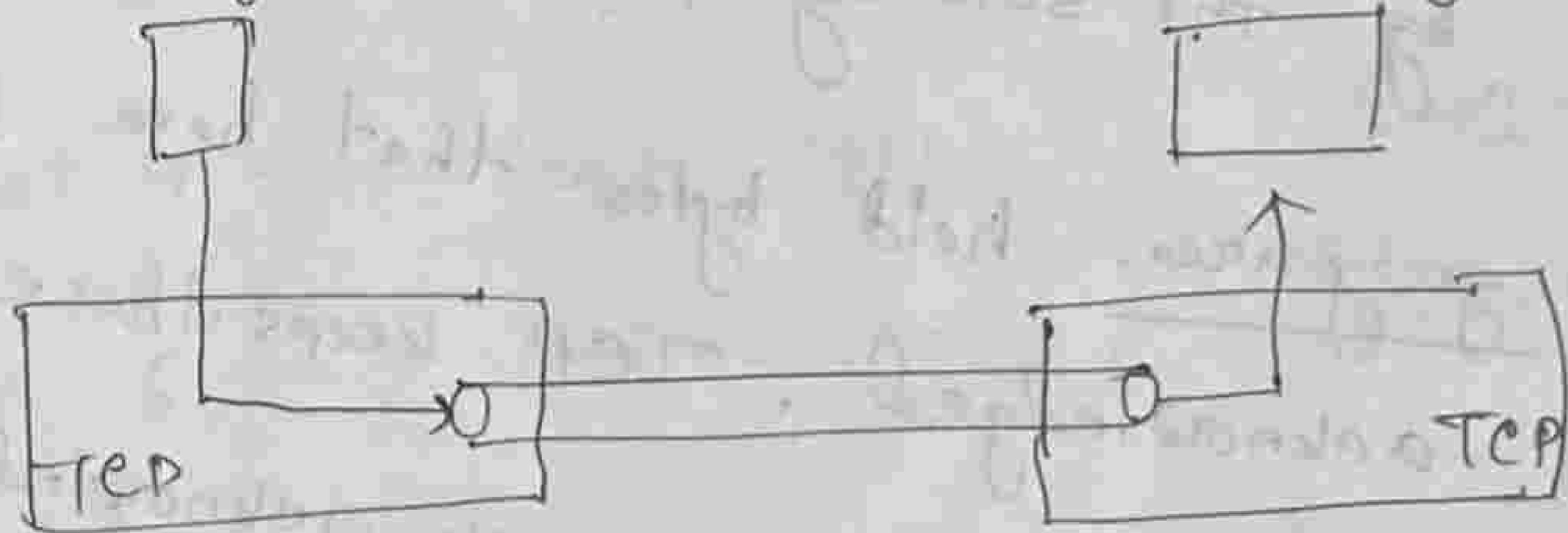
TCP Services: at the application layer

- process to process communication
- Stream delivery Service
- full duplex communication
- Connection oriented service
- Reliable service

Stream delivery Services:

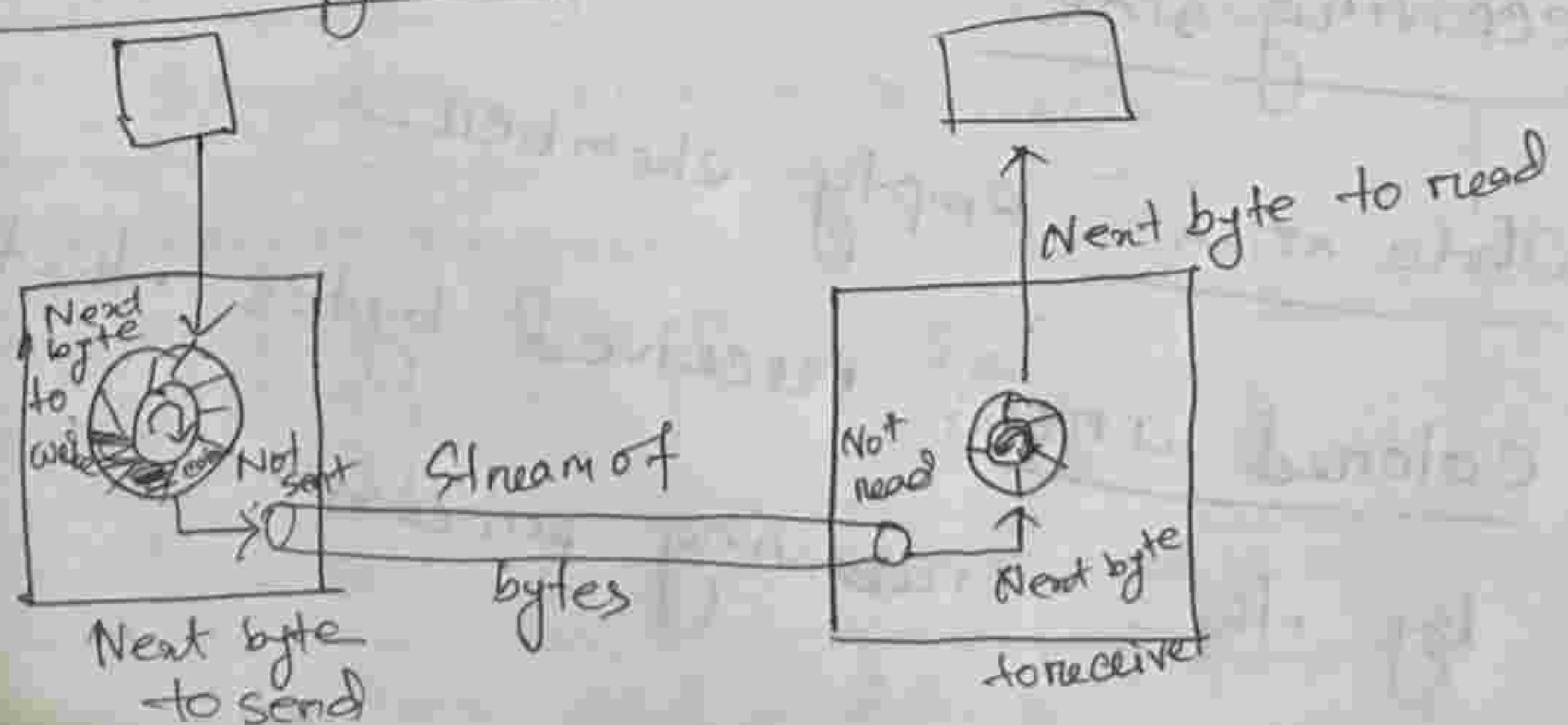
Sending process

Receiving process



- Stream of bytes → sending process/receiving process
- Two process connected by an imaginary tube.

Sending and receiving buffers:



→ the sending and receiving processes may not write or read data at the same speed.

TCP needs buffers for storage.

Two buffers: sending buffer and receiving buffer
one for each direction

Sending side:

The white section: empty chambers that can be filled by the sending process.

The gray area: hold bytes that have been sent but yet acknowledged. TCP keeps these bytes in the buffer until it receives an acknowledgement.

The colored area: bytes to be sent by sending TCP.

Receiving side:

White area: empty chamber

Colored area: received bytes that can be read by the receiving process.

no. of bytes together into a Segment

→ TCP adds a header to each segment and deliver the segment to the IP layer for transmission.

→ Segments are encapsulated in IP datagrams and transmitted.

Example:

Segment 1 → Seq no: 10001 (ranges 10001 - 11000)

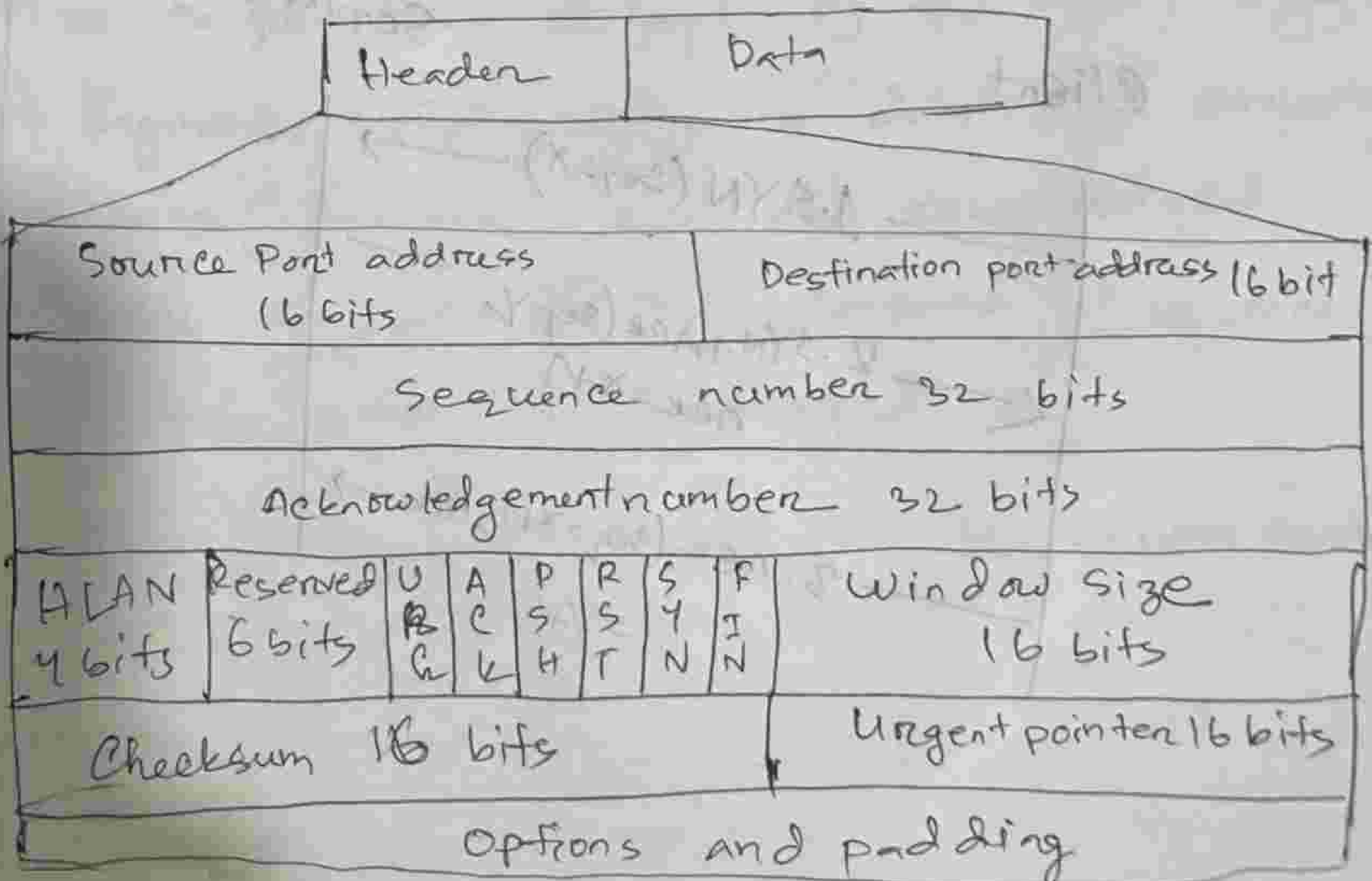
Segment 2 → Seq no: 11001 (ranges 11001 - 12000)

Segment 3 → Seq no: 12001 (ranges 12001 - 13000)

Segment 4 → Seq no: 13001 (u 13001 - 14000)

Segment 5 → Seq no: 14001 (u 14001 - 15000)

Format of TCP segment:



no. of bytes together into a Segment

→ TCP adds a header to each segment and deliver the segment to the IP layer for transmission.

→ Segments are encapsulated in IP datagrams and transmitted.

Example:

Segment 1 → Seq no: 10001 (ranges 10001 - 11000)

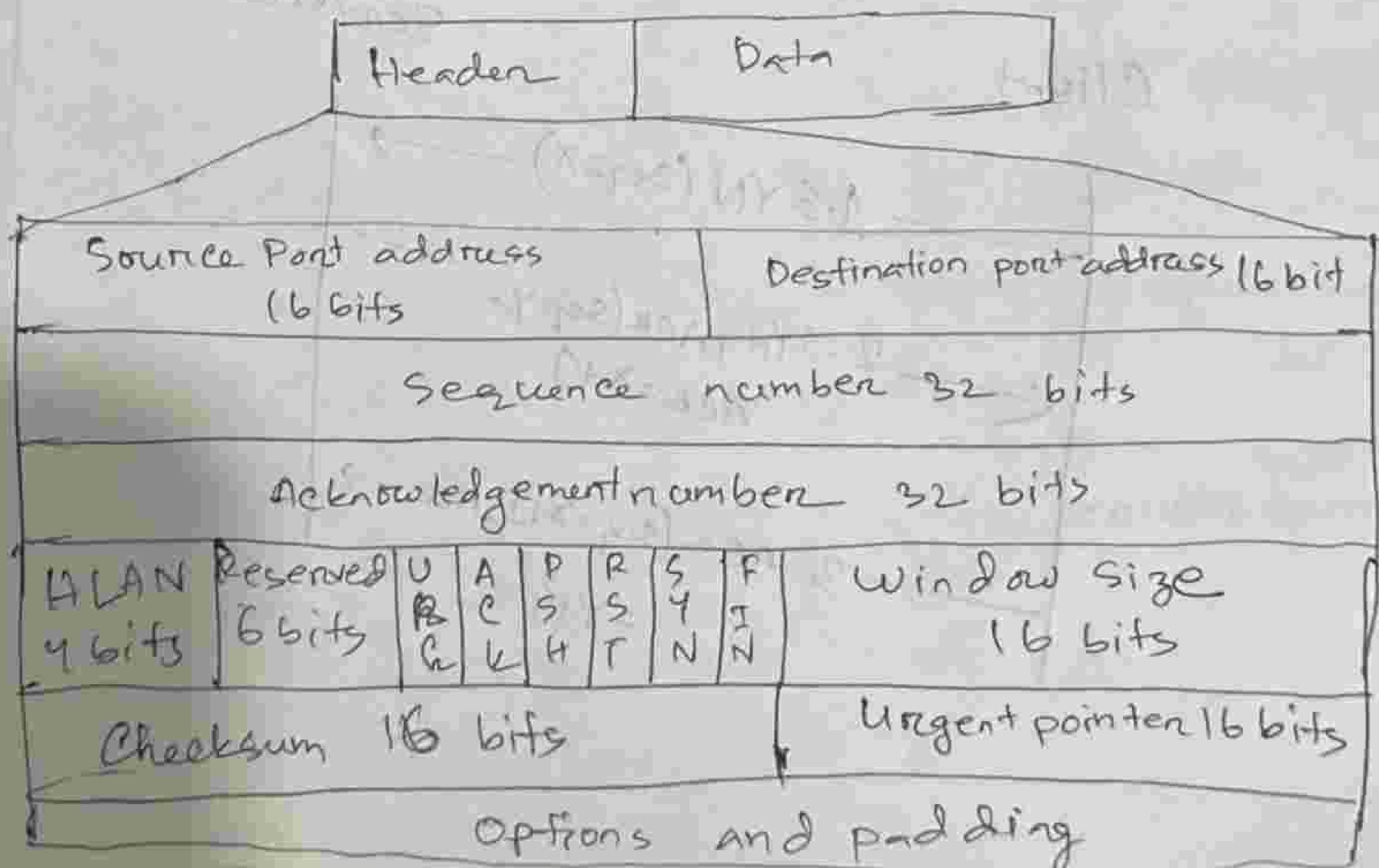
Segment 2 → Seq no: 11001 (ranges 11001 - 12000)

Segment 3 → Seq nos: 12001 (ranges 12001 - 13000)

Segment 4 → Seq no: 13001 (u 13001 - 14000)

Segment 5 → Seq no: 14001 (u 14001 - 15000)

Format of TCP segment:



URG → Urgent pointer is valid

ACK → Acknowledgement is valid

PSH → Request for push

RST → Reset the connection

SYN → Synchronize sequence number

FIN → Terminate the connection

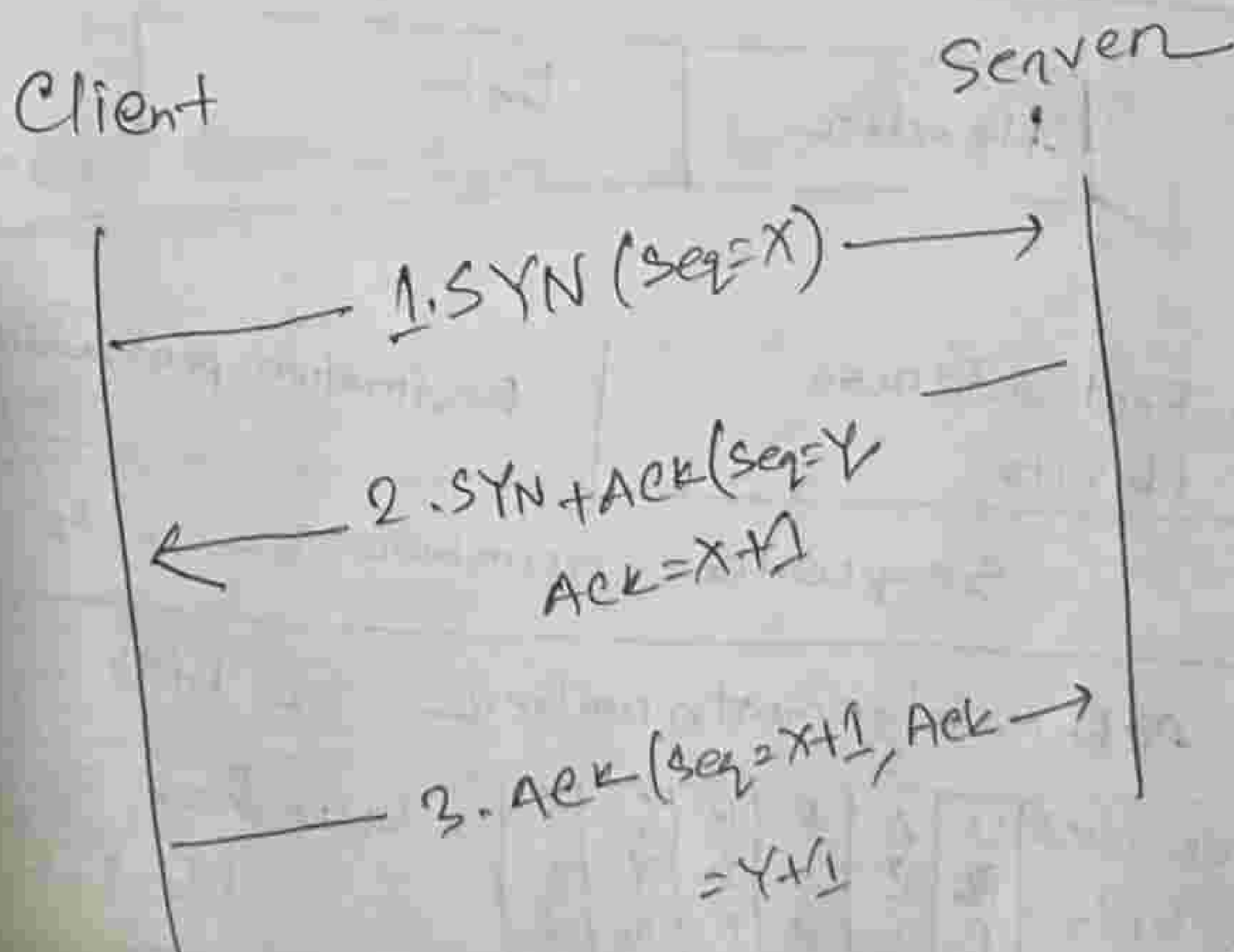
TCP connection oriented transmission ^{3 phase} ~~require~~ →

(i) Connection establishment

(ii) Data transfer

(iii) Connection termination

Three way handshaking:



Error Control:

Include mechanism for detecting corrupted segments, lost segment, out of order segment, duplicated segment.

Three simple tools — ① checksum ② acknowledgement ③ time out.

Checksum: used to check for a corrupted segment.

If the segment is corrupted it discarded by the destination TCP and considered lost.

→ Used a 16 bit checksum in each segment

Acknowledgement:

→ to confirm the receipt of the data segments

→ Segments that carry no data but consume a sequence number are also acknowledged.

→ ACK segments do not consume sequence number and are never acknowledged.

Retransmission: heart of the error control mechanism

Retrans

⇒ A segment is retransmitted on two occasions —

① When a transmission timer expires

② When sender receives three duplicate ACK segments

⇒ Retransmission after RTO:

→ Recent implementation of TCP maintains one ~~retrans~~ RTO timer for all outstanding segments. (Sent but not Acknowledged)

→ When the timer matures, the earliest outstanding segment received is retransmitted even though lack of ACK can be due to a delayed segment

→ No Timeout timer is set for a segment that carries only an acknowledgement, which means no such segment is resent

→ Value of RTO is dynamic and updated based on the round trip time of segments.

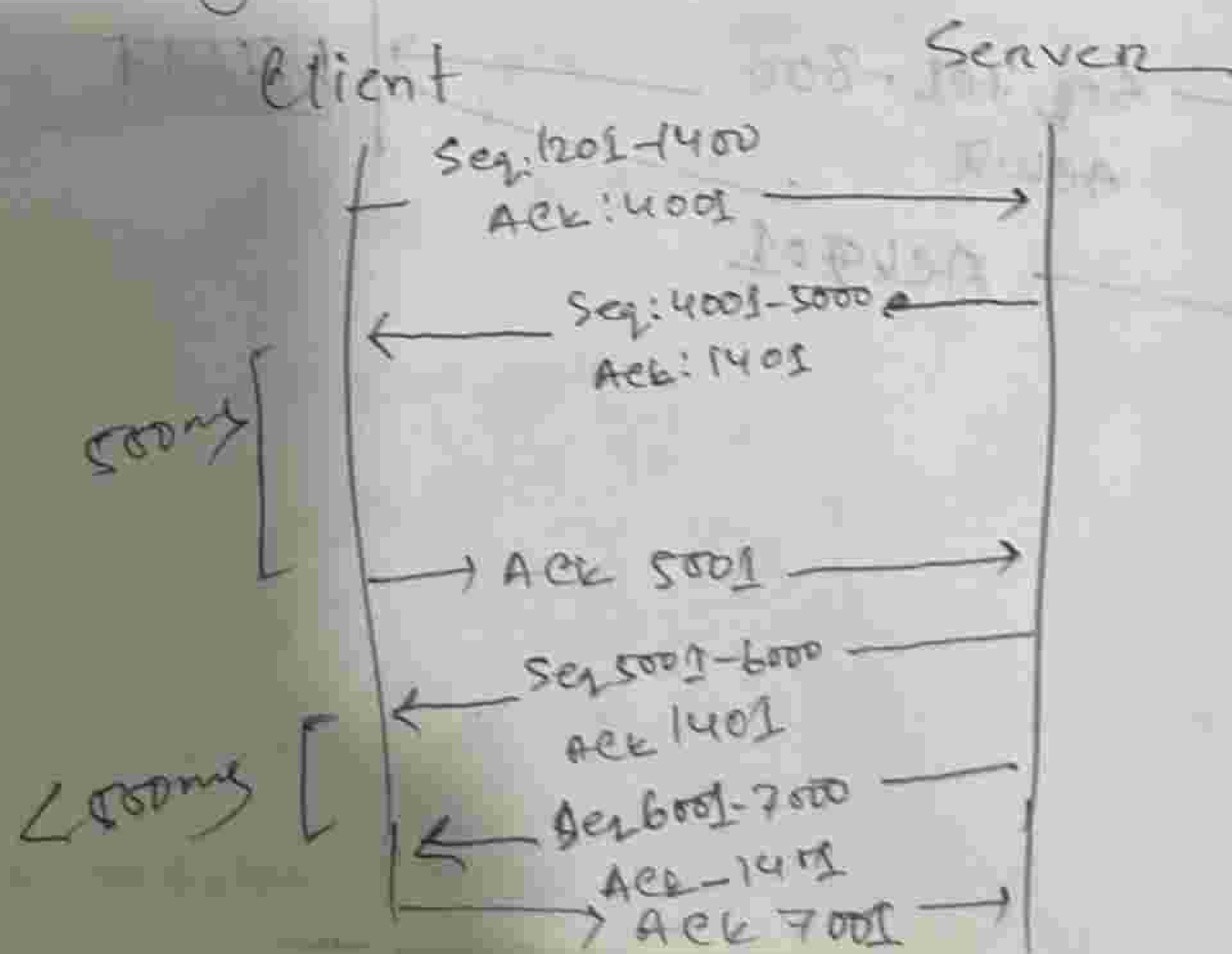
→ An RTT is the time needed for a segment to reach a destination and for an acknowledgement to be received.

Retransmission after 3 duplicate segment:-

The previous rule about Retransmission is sufficient if the value of RTO is not very large.

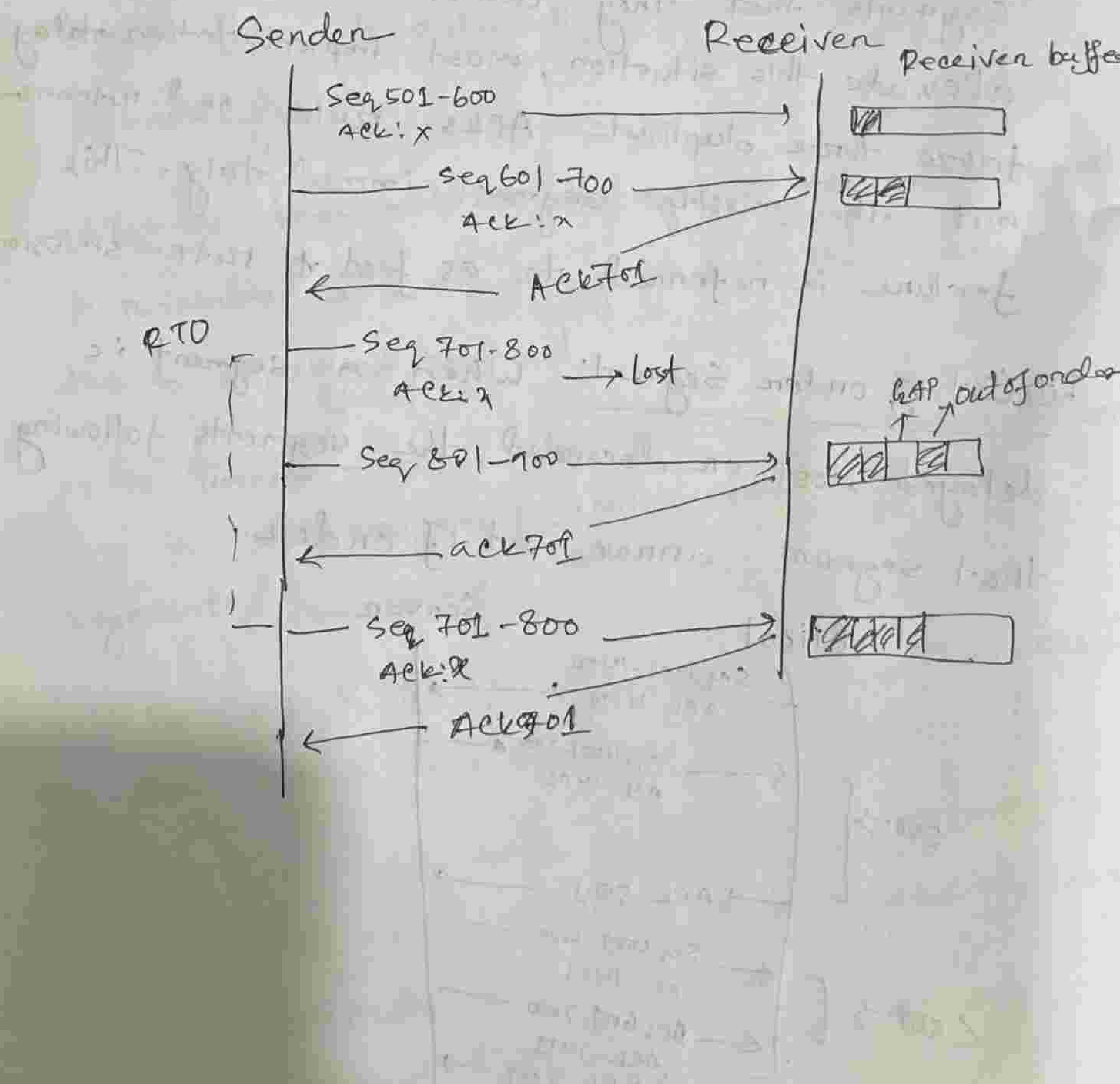
Sometimes, however, one segment is lost and receiver receives so many out of order segments that, they cannot be saved. To alleviate this situation, most implementation today follow three duplicate Acks rule and retransmit the missing segment immediately. This feature is referred to as fast retransmission.

Out of order Segment: When a segment is delayed, lost or discarded, the segments following that segment arrive out of order.



Lost Segment: treated same way as corrupted segments by the receiver.

→ A lost segment is discarded somewhere in the network; a corrupted segment is discarded by the receiver itself. Both are considered lost



☐ Congestion Control:

too many sources sending too much data too fast for network to handle.

congestion is if the number of packets sent to the network is greater than the capacity of the network

Manifestation —

→ lost packets (buffer overflow at router)

→ long delays (queuing in router buffers)

Congestion Control: refers to techniques and mechanisms that can either prevent congestion, before it happens, or remove congestion after it has happened.

Two categories

(1) Open loop congestion control

(2) Closed loop congestion control

→ Retransmission policy

→ Window policy

→ Ack policy

→ Discarding policy

→ Admission policy

Closed loop:-

- Back pressure
- Choke packet
- Implicit signaling
- Explicit signaling

Open loop:- try to prevent congestion occurring by good design.

Closed loop:- monitor the system to detect congestion pass the information where action can be taken and adjust system operation to correct the problem

TCP congestion window

Actual window size = minimum (rwnd, cwnd)

rwnd = receiver window size

cwnd = congestion window size

TCP policy to handle congestion —

~~① slow start~~ ~~② sender starts with a very slow rate~~

of transmission
~~③ increase the rate rapidly to reach a threshold.~~

① slow start ② avoidance ③ detection

The initialization step:

When a brand new connection is established the algorithm initializes its core variables to base line values —

$cwnd = 1 \text{ MSS}$ (Starts with a congestion window size of exactly one Maximum Segment Size packet)

$ss_thresh = \text{Initial value}$ (Set to high default threshold value, such as 65535 bytes to allow initial exploration)

The slow start loop:

While algorithm finds that $cwnd < ss_thresh$ it executes the slow start loop —

Every time a new, valid ACK packet arrives from the receiver,

$$cwnd = cwnd + 1$$

Because a sender can transmit a full window of packets per Round Trip Time (RTT), adding 1 to $cwnd$ for every individual ACK causes the window size to double exponentially round-trip cycle (1 → 2 → 4 → 8 ...)

Exit Condition — The moment $cwnd$ becomes equal to or greater than $ssthresh$, the loop breaks and the algorithm shifts gears to congestion avoidance.

Congestion Avoidance loop $\rightarrow$

While the algorithm finds that $cwnd \geq ssthresh$ it operates under the congestion avoidance framework.

The algorithm stops increasing $cwnd$ for every single ACK. Instead it waits for an entire Round trip transmission to complete successfully before scaling up. For every single valid ACK received it increments by a fraction

$$cwnd = cwnd + \frac{1}{cwnd}$$

Result: Summed up over an entire round where $cwnd$ number of ACK arrive, the window expands by exactly $+1$ MSS per round trip Time. This changes the growth profile to a safe linear curve ($16 \rightarrow 17 \rightarrow 18 \rightarrow 19 \dots$)

IV Detecting packet loss —

The loops above run continuously until a packet is dropped. In a network, packet drops represent congestion. The algorithm samples the type of retransmission event and handles it using one of two explicit branches —

A. If a time out occurs —

If retransmission countdown timer hit 0 before an ACK arrives, the algorithm assumes a major bottleneck. It executes a Multiplicative Decrease —

cut the threshold: update $ssthresh$ to half of the current window speed —

$$ssthresh = \frac{cwnd}{2}$$

Reset Window: Drop the transmission throttle completely back down to the safety baseline
 $cwnd = 1 MSS$

Force the state back into ^{slow start} ~~phase 1~~ to cautiously rebuild the connection from scratch

B. Three duplicates ACK arrives:—

If the sender sees 3 identical duplicate ACK in a row it means one packet went missing.

Cut the threshold: update $ss_thresh = \frac{cwnd}{2}$

Adjust window: instead of dropping to 1,
set cwnd directly to the new threshold
Value —

$$cwnd = ss_thresh$$

State Resol: Bypass slow start entirely.

